

Winograd 컨볼루션 변환: 처음부터 끝까지

목적: 3×3 컨볼루션 필터(Sobel, CNN 등)에 Winograd 변환을 적용하여 곱셈 수를 줄이는 알고리즘의 유도와 비교. 대상: 행렬곱과 컨볼루션을 아는 사람. 최종 목표: MNIST CNN 가속기에 쓸 최적의 Winograd 변환 선정.

목차

- [1. Winograd가 줄여주는 것은 무엇인가](#)
- [2. 1D Winograd \$F\(2, 3\)\$ — 직관부터](#)
- [3. 2D로 확장 — \$F\(2 \times 2, 3 \times 3\)\$](#)
- [4. Sobel 필터에 \$F\(2 \times 2, 3 \times 3\)\$ 적용](#)
- [5. 변환 행렬 \$G, A, B\$ 는 어디서 나왔는가 — Toom-Cook 알고리즘](#)
- [6. \$F\(4, 3\)\$ — 더 큰 출력 타일](#)
- [7. 복소수 점을 쓰면? — 단위근과 FFT의 등장](#)
- [8. 복소수 \$F\(4 \times 4, 3 \times 3\)\$ — 점 \$\{0, \pm 1, \pm i, \infty\}\$ 로 직접 유도](#)
- [9. 3×3 필터에 대한 Winograd 변환 비교표](#)
- [10. MNIST CNN 가속기를 위한 최종 선택](#)

1. Winograd가 줄여주는 것은 무엇인가

1.1 기본 아이디어

3×3 컨볼루션으로 2×2 출력 타일 하나를 만들려면 4×4 입력 타일이 필요하다 (3×3 윈도우를 2×2번 슬라이드).

- 직접 컨볼루션 (direct): 출력 4픽셀 × 곱셈 9 = 36번 곱셈
- Winograd $F(2 \times 2, 3 \times 3)$: 16번 곱셈 (2.25× 절감)

곱셈을 덧셈으로 바꾸는 트릭. 하드웨어에서 곱셈기는 비싸고 덧셈기는 싸므로 이득.

1.2 핵심 수식

$$Y = A^T [(G, g, G^T) \odot (B^T d B)] A$$

- d : 입력 4×4 타일
- g : 필터 3×3
- B, G, A : 상수 변환 행렬 (점 집합에서 결정됨)
- $\odot$: element-wise 곱 (같은 위치끼리 곱하기, 행렬곱 아님!)

- Y : 출력 2×2 타일

헷갈리기 쉬운 포인트: $\odot$ 는 element-wise. 행렬곱이 아님.

2. 1D Winograd F(2, 3) — 직관부터

2.1 문제 정의

입력 $d = [d_0, d_1, d_2, d_3]$, 필터 $g = [g_0, g_1, g_2]$, 출력 2개:

$$y_0 = g_0 d_0 + g_1 d_1 + g_2 d_2$$

$$y_1 = g_0 d_1 + g_1 d_2 + g_2 d_3$$

직접 계산: 곱셈 6번.

2.2 Winograd 트릭

$$m_1 = (d_0 - d_2) \cdot g_0$$

$$m_2 = (d_1 + d_2) \cdot \frac{g_0 + g_1 + g_2}{2}$$

$$m_3 = (d_2 - d_1) \cdot \frac{g_0 - g_1 + g_2}{2}$$

$$m_4 = (d_1 - d_3) \cdot g_2$$

$$y_0 = m_1 + m_2 + m_3, \quad y_1 = m_2 - m_3 - m_4$$

곱셈 4번으로 줄어듦. 계산: y_0 전개하면 $d_0 g_0 + d_1 g_1 + d_2 g_2 \checkmark$

핵심: m_2, m_3 의 $(g_0 \pm g_1 + g_2)/2$ 부분은 필터에만 의존하므로 사전 계산 가능.
Weight Stationary 패턴에 자연스럽게 맞춤.

2.3 행렬 형태

$$B^T = [1 \ 0 \ -1 \ 0 \ 0 \ 1 \ 1 \ 0 \ 0 \ -1 \ 1 \ 0 \ 0 \ 1 \ 0 \ -1]$$

$$G = [1 \ 0 \ 0 \ \frac{1}{2} \ \frac{1}{2} \ \frac{1}{2} \ \frac{1}{2} \ -\frac{1}{2} \ \frac{1}{2} \ 0 \ 0 \ 1]$$

$$A^T = [1 \ 1 \ 1 \ 0 \ 0 \ 1 \ -1 \ -1]$$

수식:

$$y = A^T [(Gg) \odot (B^T d)]$$

- $B^T d$: 입력 변환 (덧셈만)
- $G g$: 필터 변환 (덧셈 + $\times 1/2$, 사전 계산 가능)
- $\odot$: element-wise 곱 $\rightarrow$ 실제 곱셈 4번
- $A^T (\dots)$: 출력 변환 (덧셈만)

3. 2D로 확장 — F(2×2, 3×3)

1D를 두 방향으로 적용. g와 d가 행렬일 뿐.

$$U = G, g, G^T \quad (4 \times 4, \text{변환된 필터})$$

$$V = B^T, d, B \quad (4 \times 4, \text{변환된 입력})$$

$$Y = A^T, (U \odot V), A \quad (2 \times 2, \text{출력})$$

곱셈 회수: U, V 모두 $4 \times 4 \rightarrow$ element-wise 곱은 **16**번. 직접 $36 \rightarrow 16$, **2.25×** 절감.

4. Sobel 필터에 F(2×2, 3×3) 적용

4.1 Sobel Gx 변환

$$g = G_x = \begin{bmatrix} -1 & 0 & 1 & -2 & 0 & 2 & -1 & 0 & 1 \end{bmatrix}$$

Step A: Gg (4×3 행렬)

G의 행을 [a, b, c] 로 보고 Gg 의 그 행 = $a \cdot g[0, :] + b \cdot g[1, :] + c \cdot g[2, :]$:

- 행 0 [1, 0, 0] : g의 0행 = [-1, 0, 1]
- 행 1 [$\frac{1}{2}, \frac{1}{2}, \frac{1}{2}$] : $\frac{1}{2}([-1, 0, 1] + [-2, 0, 2] + [-1, 0, 1]) = [-2, 0, 2]$
- 행 2 [$\frac{1}{2}, -\frac{1}{2}, \frac{1}{2}$] : $\frac{1}{2}([-1, 0, 1] - [-2, 0, 2] + [-1, 0, 1]) = [0, 0, 0]$
- 행 3 [0, 0, 1] : g의 2행 = [-1, 0, 1]

$$Gg = \begin{bmatrix} -1 & 0 & 1 & -2 & 0 & 2 & 0 & 0 & -1 & 0 & 1 \end{bmatrix}$$

Step B: (Gg) G^T

(Gg) 의 행 [p, q, r] 에 대해 결과의 열:

- 열 0 (G의 0행 [1, 0, 0]): p
- 열 1 (G의 1행 [$\frac{1}{2}, \frac{1}{2}, \frac{1}{2}$]): $(p+q+r)/2$
- 열 2 (G의 2행 [$\frac{1}{2}, -\frac{1}{2}, \frac{1}{2}$]): $(p-q+r)/2$
- 열 3 (G의 3행 [0, 0, 1]): r

각 행에 적용:

$$U_x = GG_xG^T = \begin{bmatrix} -1 & 0 & 0 & 1 & -2 & 0 & 0 & 2 & 0 & 0 & 0 & -1 & 0 & 0 & 1 \end{bmatrix}$$

관찰: 16개 원소 중 0이 **8**개. 그리고 비영 원소는 모두 ± 1 , ± 2 . 즉 element-wise 곱에서 절반은 안 해도 되고, 나머지도 시프트와 부호 반전으로 처리됨.

4.2 Sobel Gy 변환

$$g = G_y = \begin{bmatrix} -1 & -2 & -1 & 0 & 0 & 1 & 2 & 1 \end{bmatrix}$$

같은 계산으로:

$$U_y = GG_yG^T = \begin{bmatrix} -1 & -2 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 2 & 0 & 1 \end{bmatrix}$$

0이 10개. Gx보다 더 sparse.

4.3 의미

Sobel같은 0, ± 1 , ± 2 커널에는 Winograd가 곱셈을 본질적으로 줄여주진 않는다 (원래 직접 컨볼루션도 곱셈기 안 씀, $\lll 1$ 시프트면 충분). 하지만 **Winograd**가 어떻게 적용되는지 보여주는 좋은 연습 대상.

Winograd가 진짜 빛나는 곳: 임의 가중치를 가진 **CNN** 필터 (VGG, ResNet, MNIST CNN 등).

5. 변환 행렬 G, A, B는 어디서 나왔는가 — Toom-Cook 알고리즘

5.1 컨볼루션 = 다항식 곱셈

1D 컨볼루션을 다항식 곱으로 본다. 필터를 뒤집어서 다항식으로 표현:

$$g(x) = g_2 + g_1x + g_0x^2$$

$$d(x) = d_0 + d_1x + d_2x^2 + d_3x^3$$

$s(x) = g(x) \cdot d(x)$ 의 계수를 계산하면:

- s_2 계수 = $g_2 d_2 + g_1 d_1 + g_0 d_0 = \mathbf{y_0}$ ✓
- s_3 계수 = $g_2 d_3 + g_1 d_2 + g_0 d_1 = \mathbf{y_1}$ ✓

핵심 통찰: "출력 m개, 필터 길이 r" 컨볼루션 = "차수 (r-1) × 차수 (m+r-2) 다항식 곱"에서 가운데 m개 계수 뽑기.

5.2 점에서 평가하면 곱셈이 줄어든다

라그랑주 정리: 차수 N인 다항식은 서로 다른 점 N+1개에서의 값으로 완전히 결정된다.

$s(x)$ 는 차수 (m+r-2). 따라서 점 (**m+r-1**)개에서 평가하면 $s(x)$ 복원 가능.

전략:

- 점 (m+r-1)개 $\alpha_0, \alpha_1, \dots, \alpha_{\{m+r-2\}}$ 고르기
- $g(\alpha_i), d(\alpha_i)$ 계산 (덧셈만, g 쪽은 사전 계산)
- $s(\alpha_i) = g(\alpha_i) \cdot d(\alpha_i) \leftarrow$ 여기서 곱셈 (**m+r-1**)번
- 라그랑주 보간으로 원하는 m개 계수 복원 (덧셈만)

F(2, 3): m+r-1 = 4점 필요. 직접 6번 → Winograd 4번.

5.3 점은 어떻게 고를까?

수학적으로는 어떤 점이든 OK. 하드웨어에선 곱셈/나눗셈 없는 점을 선호.

표준 선택 (F(2,3)):

$$\alpha_0 = 0, \quad \alpha_1 = 1, \quad \alpha_2 = -1, \quad \alpha_3 = \infty$$

∞ 는 "최고차 계수만 본다"는 의미. $g(\infty) = g_2$, $d(\infty) = d_3$. 변환 행렬에 분수가 안 생겨 깔끔해짐.

5.4 G, B^T, A^T가 어디서 나오는지

G (4×3) = 필터 g를 4개 점에서 평가:

점 α	$g(\alpha)$ (필터 뒤집은 표기)	G의 행
0	g_0	[1, 0, 0]
1	$(g_0 + g_1 + g_2) / 2$ (정규화 1/2)	[1/2, 1/2, 1/2]
-1	$(g_0 - g_1 + g_2) / 2$	[1/2, -1/2, 1/2]
∞	g_2	[0, 0, 1]

1/2은 라그랑주 보간 분모에서 나오는 정규화. G에 미리 흡수하여 A를 정수로 만든다.

B^T (4×4) = 입력 d를 4개 점에서 평가하는 변환.

A^T (2×4) = 4개 점 평가값으로부터 원하는 출력 계수 (s_2, s_3) 복원하는 라그랑주 보간 가중치.

5.5 일반 절차 — Cook-Toom 알고리즘

Winograd F(m, r) 만들기 = 출력 m, 필터 r → 점 (m+r-1)개 고르고 평가-보간

- 점 집합 $\{\alpha_0, \dots, \alpha_{m+r-2}\}$ 선택 (보통 $\{0, \pm 1, \pm 2, \infty\}$ 등에서)
- $g(\alpha_i)$, $d(\alpha_i)$ 표 작성 → G와 B^T가 나옴
- 라그랑주 보간 → A^T가 나옴

6. F(4, 3) — 더 큰 출력 타일

6.1 왜 m을 키우고 싶은가

F(m, 3)	직접 곱셈	Winograd 곱셈	1D 절감	2D 절감 (m×m)
F(2, 3)	6	4	1.5×	2.25×
F(3, 3)	9	5	1.8×	3.24×
F(4, 3)	12	6	2.0×	4.00×
F(6, 3)	18	8	2.25×	5.06×

m이 클수록 절감 큼. 하지만...

6.2 m이 커질수록 나빠지는 것들

1. 변환 행렬에 분수 등장: $F(4,3)$ 은 $\pm 1/24$, $F(6,3)$ 은 $\pm 1/720$ 등
2. 수치 정밀도 손실: 양자화된 CNN에 치명적
3. 변환 가산기 비용 증가: 곱셈은 줄지만 덧셈은 늘어남
4. 라인버퍼/메모리 요구량 증가

6.3 F(4, 3) 표준 (실수 점)

점 집합: $\{0, 1, -1, 2, -2, \infty\}$ (6점)

곱셈: 6번 (직접 $12 \rightarrow 6$, 50% 감소)

행렬:

$$B^T = \begin{bmatrix} 4 & 0 & -5 & 0 & 1 & 0 & 0 & -4 & -4 & 1 & 1 & 0 & 0 & 4 & -4 & -1 & 1 & 0 & 0 & -2 & -1 & 2 & 1 & 0 & 0 & ; \end{bmatrix}$$

원소: $\{0, \pm 1, \pm 2, \pm 4, \pm 5\}$ — 모두 정수, 시프트와 덧셈으로 처리

$$G = \begin{bmatrix} \frac{1}{4} & 0 & 0 & -\frac{1}{6} & -\frac{1}{6} & -\frac{1}{6} & -\frac{1}{6} & \frac{1}{6} & -\frac{1}{6} & \frac{1}{24} & \frac{1}{12} & \frac{1}{6} & \frac{1}{24} & -\frac{1}{12} & \frac{1}{6} & 0 & 0 & 1 \end{bmatrix}$$

⚠ **1/24, 1/12, 1/6** 등장. 그러나 G는 사전 계산되므로 런타임에 보이지 않음.

$$A^T = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 0 & 0 & 1 & -1 & 2 & -2 & 0 & 0 & 1 & 1 & 4 & 4 & 0 & 0 & 1 & -1 & 8 & -8 & 1 \end{bmatrix}$$

원소: $\{0, \pm 1, \pm 2, \pm 4, \pm 8\}$ — 모두 정수

6.4 정밀도 문제

	FP32	FP16	INT16	INT8
F(4×4, 3×3)	무손실	약간 손실	측정 가능 손실	심각한 손실

이유: G의 $1/24$ 가 양자화 그리드에 떨어지지 않음. INT8에서 누적 오차 발생.

INT8 양자화된 CNN에서는 실수 F(4,3) 부적합.

7. 복소수 점을 쓰면? — 단위근과 FFT의 등장

7.1 동기

실수 F(4,3)의 분수 문제를 해결하고 싶다. 점 사이 거리가 균일하면 라그랑주 분모가 작아진다.

관찰: 4차 단위근 $\{1, i, -1, -i\}$ 는 서로 거리가 모두 $\sqrt{2}$ 로 균일.

7.2 점 집합 후보

$\{0, 1, -1, i, -i, \infty\}$ (6점)

- 단위근 4개 + $0 + \infty$
- $F(4,3)$ 에 필요한 6점에 정확히 맞음

7.3 평가 시 일어나는 일

실수 필터/입력을 복소수 점에서 평가하면 켤레 쌍이 자동으로 발생:

$$g(i) = \overline{g(-i)}, \quad d(i) = \overline{d(-i)} \\ \Rightarrow s(i) = \overline{s(-i)}$$

→ i 점에서만 계산하면 $-i$ 점은 공짜 (켤레)

이게 FFT에서 실수 신호를 절반의 비용으로 처리하는 원리와 동일.

8. 복소수 $F(4 \times 4, 3 \times 3)$ — 점 $\{0, \pm 1, \pm i, \infty\}$ 로 직접 유도

이게 핵심 챕터.

8.1 셋업

- 필터: $g(x) = g_0 + g_1 x + g_2 x^2$ (차수 2)
- 입력: $d(x) = d_0 + d_1 x + d_2 x^2 + d_3 x^3 + d_4 x^4 + d_5 x^5$ (차수 5)
- 곱: $s(x) = g(x) \cdot d(x)$ (차수 7)
- 점 6개로 차수 5 다항식까지 정확히 복원 가능
 - 필터를 뒤집어서 정의하면 출력 $y_0 \sim y_3 = s(x)$ 의 x^2, x^3, x^4, x^5 계수

8.2 필터 평가 → G 행렬

$g(x) = g_0 + g_1 x + g_2 x^2$ 평가:

점	$g(\text{점})$	G 행
0	g_0	$[1, 0, 0]$
1	$g_0 + g_1 + g_2$	$[1, 1, 1]$
-1	$g_0 - g_1 + g_2$	$[1, -1, 1]$
i	$(g_0 - g_2) + i \cdot g_1$	$[1, i, -1]$
$-i$	$(g_0 - g_2) - i \cdot g_1$	$[1, -i, -1]$
∞	g_2	$[0, 0, 1]$

$$G = [1 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ -1 \ 1 \ i \ -1 \ -i \ -1 \ 0 \ 0 \ 1]$$

원소: {0, ±1, ±i} — Gaussian integer만! ✅ 분수 0개.

8.3 입력 평가 → B^T 행렬

$d(x) = d_0 + d_1 x + \dots + d_5 x^5$ 평가:

점	d(점)
0	d_0
1	$d_0+d_1+d_2+d_3+d_4+d_5$
-1	$d_0-d_1+d_2-d_3+d_4-d_5$
i	$(d_0-d_2+d_4) + i(d_1-d_3+d_5)$
-i	$(d_0-d_2+d_4) - i(d_1-d_3+d_5)$
∞	d_5

$$B^T = [1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ -1 \ 1 \ -1 \ 1 \ -1 \ i \ -1 \ -i \ 1 \ i \ 1 \ -i]$$

원소: {0, ±1, ±i} ✅ 분수 0개.

8.4 라그랑주 보간으로 A^T 구하기

각 점 α_k 의 라그랑주 기저 다항식:

$$L_k(x) = \prod_{j \neq k} \frac{x - \alpha_j}{\alpha_k - \alpha_j}$$

분모 계산 (각 α_k 에 대해 다른 점들과의 거리 곱):

α_k	분모 계산	결과
0	$(0-1)(0+1)(0-i)(0+i) = (-1)(1)(-i)(i) = (-1)(1)(1)$	-1
1	$(1-0)(1+1)(1-i)(1+i) = 1 \cdot 2 \cdot (1-i^2) = 2 \cdot 2$	4
-1	$(-1)(-2)(-1-i)(-1+i) = 2 \cdot ((-1)^2-i^2) = 2 \cdot 2$	4
i	$i(i-1)(i+1)(2i) = i(i^2-1)(2i) = i(-2)(2i) = -4i^2$	4
-i	(컬레)	4
∞	(∞ 특별 처리)	1

핵심: 모든 분모가 {1, 4} . 깔끔한 2의 거듭제곱.

분자 다항식 (∞ 점은 인자 제외, 차수 1 감소):

L_0(x), α_0 = 0, 분모 -1:

$$\psi_0(x) = (x-1)(x+1)(x-i)(x+i) = (x^2-1)(x^2+1) = x^4-1$$

$$L_0(x) = (x^4-1)/(-1) = 1-x^4$$

L_1(x), α_1 = 1, 분모 4:

$$\psi_1(x) = x(x+1)(x-i)(x+i) = x(x+1)(x^2+1) = x^4+x^3+x^2+x$$

$$L_1(x) = (x^4+x^3+x^2+x)/4$$

L_2(x), α_2 = -1, 분모 4:

$$\psi_2(x) = x(x-1)(x^2+1) = x^4-x^3+x^2-x$$

$$L_2(x) = (x^4-x^3+x^2-x)/4$$

L_3(x), α_3 = i, 분모 4:

$$\psi_3(x) = x(x-1)(x+1)(x+i) = x(x^2-1)(x+i)$$

$$(x^2-1)(x+i) = x^3+ix^2-x-i$$

$$x \cdot (x^3+ix^2-x-i) = x^4+ix^3-x^2-ix$$

$$L_3(x) = (x^4+ix^3-x^2-ix)/4$$

L_4(x), α_4 = -i, 분모 4 (켈레):

$$L_4(x) = (x^4-ix^3-x^2+ix)/4$$

검증: L_3 + L_4 (켈레 합)는 실수여야 함.

$$L_3 + L_4 = (2x^4 - 2x^2)/4 = (x^4 - x^2)/2 \quad \checkmark$$

L_5(x), α_5 = ∞, 분모 1: ∞에서 1, 다른 모든 점에서 0이어야 함. leading 계수 1.

$$L_5(x) = x(x-1)(x+1)(x-i)(x+i) = x(x^4-1) = x^5-x$$

8.5 각 L_k의 계수 표

A^T[j, k] = L_k(x)의 x^(j+2) 계수. (j=0,1,2,3은 출력 y_0~y_3 대응, s(x)의 x^2, x^3, x^4, x^5 계수)

k	L_k(x)	x ² 계수	x ³ 계수	x ⁴ 계수	x ⁵ 계수
0	1 - x ⁴	0	0	-1	0
1	(x ⁴ +x ³ +x ² +x)/4	1/4	1/4	1/4	0
2	(x ⁴ -x ³ +x ² -x)/4	1/4	-1/4	1/4	0
3	(x ⁴ +ix ³ -x ² -ix)/4	-1/4	i/4	1/4	0
4	(x ⁴ -ix ³ -x ² +ix)/4	-1/4	-i/4	1/4	0
5	x ⁵ - x	0	0	0	1

8.6 A^T 행렬

$$A^T = \begin{bmatrix} 0 & \frac{1}{4} & \frac{1}{4} & -\frac{1}{4} & -\frac{1}{4} & 0 & 0 & \frac{1}{4} & -\frac{1}{4} & \frac{i}{4} & -\frac{i}{4} & 0 & -1 & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

원소: $\{0, \pm 1, \pm \frac{1}{4}, \pm i/4\}$. $\frac{1}{4}$ 한 종류만 등장.

8.7 $\frac{1}{4}$ 흡수 트릭

$\frac{1}{4}$ 은 시프트 2번이지만, 완전히 정수화하려면 정규화 재배치:

$$G' = \frac{1}{4}G, \quad A'^T = 4 \cdot A^T$$

그러면:

$$G' = \frac{1}{4} \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 1 & -1 & 1 & 1 & i & -1 & -i & -1 & 0 & 0 & 1 \end{bmatrix}$$

$$A'^T = \begin{bmatrix} 0 & 1 & 1 & -1 & -1 & 0 & 0 & 1 & -1 & i & -i & 0 & -4 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 4 \end{bmatrix}$$

- G' 는 사전 계산 $\rightarrow \frac{1}{4}$ 은 가중치 전처리에서 흡수, 런타임에 보이지 않음
- A'^T 원소: $\{0, \pm 1, \pm i, \pm 4\} \rightarrow$ 시프트만으로 처리, 곱셈기 0개 

8.8 곱셈 수 (1D)

6개 점에서 $s(\alpha_k) = g(\alpha_k) \cdot d(\alpha_k)$ 계산:

- 점 $\{0, 1, -1, \infty\}$ (4개): 실수 곱셈 각 1번 = 4번
- 점 $\{i, -i\}$ (켈레쌍): 한 번만 계산하면 됨
 - 복소수 곱셈 1번 = 실수 곱셈 3번 (Gauss trick) 또는 4번 (naive)
- 3번

총 1D 실수 곱셈: $4 + 3 = 7$ 번

비교:

- 직접 1D F(4,3): 12번
- 실수 F(4,3): 6번
- 복소수 F(4,3): 7번 (절감률 $12/7 \approx 1.71\times$)

8.9 2D 확장 — 켈레 대칭 활용

$$U = G, g, G^T \quad (6 \times 6)$$

$$V = B^T, d, B \quad (6 \times 6)$$

$$Y = A^T, (U \odot V), A \quad (4 \times 4)$$

2D에서 점은 (α_i, β_j) 쌍, $6 \times 6 = 36$ 개.

켈레 관계로 곱셈 그룹 정리:

점 그룹	개수	곱셈 종류	실수 곱셈
순수 실수 점 $\{0, 1, -1, \infty\}^2$	16	실수 $\times$ 실수	16
$\{i, -i\} \times \{0, 1, -1, \infty\}$	4 (컬레 페어)	복소수 $\times$ 실수	$4 \times 3 = 12$
$\{0, 1, -1, \infty\} \times \{i, -i\}$	4 (컬레 페어)	실수 $\times$ 복소수	$4 \times 3 = 12$
$\{i, -i\}^2$	2 (컬레 페어)	복소수 $\times$ 복소수	$2 \times 3 = 6$

총 2D 실수 곱셈: $16 + 12 + 12 + 6 = 46$ 번

비교:

- 직접 2D: 144번
- 실수 F(4×4, 3×3): 36번
- 복소수 F(4×4, 3×3): 46번 (절감률 $144/46 \approx 3.13\times$)

9. 3×3 필터에 대한 Winograd 변환 비교표

9.1 점 집합과 행렬 원소

	F(2×2, 3×3)	실수 F(4×4, 3×3)	복소수 F(4×4, 3×3)
점 집합	$\{0, \pm 1, \infty\}$	$\{0, \pm 1, \pm 2, \infty\}$	$\{0, \pm 1, \pm i, \infty\}$
점 개수	4	6	6
G 원소	$\{0, \pm 1, \pm \frac{1}{2}\}$	$\{0, \pm 1/24, \pm 1/12, \pm 1/6, \pm 1/4\}$	$\{0, \pm 1, \pm i\} \times \frac{1}{4}$ 스케일
B^T 원소	$\{0, \pm 1\}$	$\{0, \pm 1, \pm 2, \pm 4, \pm 5\}$	$\{0, \pm 1, \pm i\}$
A^T 원소	$\{0, \pm 1\}$	$\{0, \pm 1, \pm 2, \pm 4, \pm 8\}$	$\{0, \pm 1, \pm i, \pm 4\}$ (¼ 흡수 후)
분수 등장	½ 1개 (G)	1/24까지 (G)	없음 (G에 ¼ 흡수)

9.2 곱셈 수

	1D 직접	1D Winograd	2D 직접	2D Winograd	2D 절감률
F(2×2, 3×3)	6	4	36	16	2.25×
실수 F(4×4, 3×3)	12	6	144	36	4.00×
복소수 F(4×4, 3×3)	12	7	144	46	3.13×

9.3 픽셀당 비용

	출력 픽셀/타일	곱셈/픽셀	가산기/픽셀 (추정)
직접 3×3 conv	1	9	8
F(2×2, 3×3)	4	4.0	~14
실수 F(4×4, 3×3)	16	2.25	~10.5
복소수 F(4×4, 3×3)	16	2.88	~8

9.4 정밀도 친화성

	FP32	FP16	INT16	INT8
F(2×2, 3×3)	✓	✓	✓	✓
실수 F(4×4, 3×3)	✓	⚠	⚠	✗
복소수 F(4×4, 3×3)	✓	✓	✓	✓

이유: 변환 행렬이 양자화 그리드에 떨어지는가의 문제. 복소수는 Gaussian integer만 등장하여 무손실.

9.5 하드웨어 자원 (FPGA 추정, 단일 채널)

	DSP48	LUT (변환+컨트롤)	BRAM (라인버퍼)	라인버퍼 행 수
F(2×2, 3×3)	16	~3K	3	3
실수 F(4×4, 3×3)	36	~8K	5	5
복소수 F(4×4, 3×3)	46 (또는 Gauss trick으로 더 절감)	~6K	5	5

9.6 종합 점수표

기준	F(2,3)	실수 F(4,3)	복소수 F(4,3)
곱셈 절감	★★★★	★★★★★	★★★★★
변환 단순성	★★★★★	★★	★★★★★
INT8 양자화 친화성	★★★★★	★	★★★★★
라인버퍼/메모리	★★★★★	★★★	★★★
구현 난이도 (학부 과제)	★★★★★	★★★	★★
Throughput	★★	★★★★★	★★★★★

10. MNIST CNN 가속기를 위한 최종 선택

10.1 MNIST 환경 특성

- 입력: 28×28 그레이스케일
- 보통 INT8 양자화로 가속
- 3×3 conv가 표준
- 타일링:
 - F(2,3): 26/2 = 13 타일 (깔끔)
 - F(4,3): 28/4 = 7 타일 (입력 6×6 타일 49개, 매우 깔끔)

10.2 권장 결론

MNIST + INT8 + 곱셈 최대 절감을 동시에 원한다면:

→ 복소수 **F(4×4, 3×3)** 가 최적

- 곱셈 144 → 46 (3.13× 절감)
- 변환 행렬 정수성 보존 → 양자화 무손실
- 라인버퍼 5행, 타일 7×7 = 49개로 28×28 깔끔 처리
- 변환이 wire swap + 덧셈만이라 LUT 효율적

학부 과제 수준에서 안정 구현이 우선이면:

→ **F(2×2, 3×3)** 추천

- 구현 난이도 가장 낮음
- 디버깅 용이
- 절감률 2.25×도 충분히 인상적
- 보고서에 정확도/파형 검증 수월

단순 곱셈 최소가 목표라면:

→ 실수 F(4×4, 3×3). 단, INT8 양자화는 포기.

10.3 보고서/발표용 한 문장

MNIST와 같이 작은 정수 입력에 대한 CNN 가속을 위해, 점 집합 $\{0, \pm 1, \pm i, \infty\}$ 을 사용하는 복소수 Winograd F(4×4, 3×3) 변환을 도입한다. 표준 실수 변환의 G 행렬이 1/12, 1/24와 같은 정밀도 손실을 야기하는 분수를 포함하는 반면, 복소수 변환은 G, B, A 모두 Gaussian integer $\{0, \pm 1, \pm i\}$ (및 G에 흡수 가능한 1/4 스케일링)만으로 구성되어 INT8 양자화 환경에서 무손실 변환이 가능하다. 쉼표 대칭을 활용한 실제 곱셈 수는 46회로, 직접 컨볼루션 144회 대비 3.13× 감소한다.

부록: 핵심 공식 요약

A.1 일반 Winograd 변환

$$1D: y = A^T [(Gg) \odot (B^T d)]$$

$$2D: Y = A^T [(GgG^T) \odot (B^T dB)] A$$

A.2 Cook-Toom 절차

F(m, r) 만들기:

1. 점 (m+r-1)개 선택
2. G: 각 점에서 g(α) 평가 → (m+r-1) × r 행렬
3. B^T: 각 점에서 d(α) 평가 → (m+r-1) × (m+r-1) 행렬
4. A^T: 라그랑주 보간으로 원하는 m개 계수 추출 → m × (m+r-1) 행렬

A.3 곱셈 수 공식

차원	직접	Winograd
1D	$m \times r$	$m + r - 1$
2D	$m^2 \times r^2$	$(m + r - 1)^2$

A.4 복소수 F(4×4, 3×3) 최종 행렬

$$G = \frac{1}{4} [1 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ -1 \ 1 \ 1 \ i \ -1 \ 1 \ -i \ -1 \ 0 \ 0 \ 1] \quad (\text{사전 계산})$$

$$B^T = [1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ -1 \ 1 \ -1 \ 1 \ -1 \ 1 \ i \ -1 \ -i \ 1 \ i \ 1 \ -i]$$

$$A^T = [0 \ 1 \ 1 \ -1 \ -1 \ 0 \ 0 \ 1 \ -1 \ i \ -i \ 0 \ -4 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 4]$$

곱셈 수: 1D 7번, 2D 46번 (컬레 대칭 활용).

참고

- Winograd, S. (1980). *Arithmetic Complexity of Computations*. SIAM.
- Lavin, A. & Gray, S. (2016). *Fast Algorithms for Convolutional Neural Networks*. CVPR.
- Toom-Cook multiplication:
https://en.wikipedia.org/wiki/Toom%E2%80%93Cook_multiplication